

WEEK 5: GRAPH THINKING

What Is a Graph Database?

A **graph database** stores data as **nodes**, **relationships**, and **properties**.

This week focuses on the **property graph model**, which is the model used by Neo4j and the model Apache AGE brings into PostgreSQL with Cypher.

Core concepts:

- Node (vertex): entity such as a person, movie, company, or city
- Relationship (edge): directed connection between nodes
- Label or type: classification, such as `Person` , `Movie` , or `ACTED_IN`
- Property: key-value attribute on a node or relationship
- Path: sequence of connected nodes and relationships

Visual example:

```
(Tom Hanks) -[:ACTED_IN]->(Forrest Gump) <-[:DIRECTED]-(Robert Zemeckis)
```

Why graphs matter:

- Relationships are first-class data, not just foreign keys
- Traversal is expressed as path matching
- Multi-hop questions become easier to write and reason about
- Common in recommendations, fraud, knowledge graphs, and network analysis

Property Graph Data Model

From the official Neo4j property-graph model:

- Nodes can have zero or more labels
- Relationships always have a direction
- Relationships have exactly one type
- Nodes and relationships can both have properties
- Paths are ordered traversals through the graph

Example graph:

```
CREATE (:Person {name: 'Tom Hanks', born: 1956})  
  -[:ACTED_IN {roles: ['Forrest']}]->  
    (:Movie {title: 'Forrest Gump', released: 1994})  
  <-[:DIRECTED]-  
    (:Person {name: 'Robert Zemeckis', born: 1951})
```

Modelling rule:

- Put entities in nodes.
- Put actions or connections in relationships.
- Put descriptive attributes in properties.

How Graph Data Looks in Relational Tables

Before using a graph database, it is useful to see the same data as tables.

Nodes table:

```
CREATE TABLE graph_nodes (  
  id INT PRIMARY KEY, label VARCHAR(100), node_type VARCHAR(50));  
INSERT INTO graph_nodes VALUES  
  (1, 'Alice', 'Person'), (2, 'Bob', 'Person'),  
  (3, 'TechCorp', 'Company'), (4, 'NYC', 'City');
```

Edges table:

```
CREATE TABLE graph_edges (  
    id INT PRIMARY KEY, from_node INT, to_node INT, relationship VARCHAR(50));  
INSERT INTO graph_edges VALUES  
    (1,1,2,'KNOWS'),(2,1,3,'WORKS_AT'),(3,3,4,'HAS_OFFICE');
```

graph_nodes:

id	label	node_type
1	Alice	Person
2	Bob	Person
3	TechCorp	Company
4	NYC	City

graph_edges:

id	from_node	to_node	relationship
1	1	2	KNOWS
2	1	3	WORKS_AT
3	3	4	HAS_OFFICE

Adjacency Lists and Their Limits

An adjacency-list design works well for storing graph-like data in SQL, but traversal quickly becomes verbose.

Find who Alice knows:

```
SELECT n.label
FROM graph_edges e
JOIN graph_nodes n ON e.to_node = n.id
WHERE e.from_node = (SELECT id FROM graph_nodes WHERE label = 'Alice')
AND e.relationship = 'KNOWS';
```

```
knows_person
-----
Bob
(1 row)
```

Find where Alice likely is:

```
SELECT city.label AS inferred_city
FROM graph_edges e1
JOIN graph_edges e2 ON e1.to_node = e2.from_node
JOIN graph_nodes city ON e2.to_node = city.id
WHERE e1.from_node = (SELECT id FROM graph_nodes WHERE label = 'Alice')
      AND e1.relationship = 'WORKS_AT' AND e2.relationship = 'HAS_OFFICE';
```

```
inferred_city
-----
NYC
(1 row)
```

The issue:

- 1 hop = manageable
- 2 hops = still okay
- 3+ hops = increasingly awkward SQL
- Unknown path length = much harder

Recursive CTEs - Foundation

A **recursive CTE** repeatedly expands a result until no new rows are found.

Syntax:

```
WITH RECURSIVE cte_name AS (  
    SELECT ...           -- anchor query  
    UNION ALL  
    SELECT ... FROM ... -- recursive: expand one step  
    JOIN cte_name ON ...  
)  
SELECT * FROM cte_name;
```

Mental model:

1. Start somewhere.
2. Expand one step.
3. Repeat until you cannot expand further.

Best practice:

- Track depth.
- Track visited nodes if cycles are possible.
- Add a depth limit for safety.

Recursive CTE Example - Hierarchies

Use case: organization chart

```
CREATE TABLE employees (  
  id INT PRIMARY KEY, name VARCHAR(255),  
  manager_id INT REFERENCES employees(id));  
INSERT INTO employees VALUES  
  (1, 'Alice', NULL), (2, 'Bob', 1), (3, 'Charlie', 1),  
  (4, 'David', 2), (5, 'Eve', 2), (6, 'Frank', 3);
```

What the table looks like:

id	name	manager_id
1	Alice	null
2	Bob	1
3	Charlie	1
4	David	2
5	Eve	2
6	Frank	3

Find all subordinates of Alice:

```
WITH RECURSIVE subordinates AS (  
  SELECT id, name, manager_id, 1 AS depth  
  FROM employees  
  WHERE manager_id = (SELECT id FROM employees WHERE name = 'Alice')  
  UNION ALL  
  SELECT e.id, e.name, e.manager_id, s.depth + 1  
  FROM employees e  
  JOIN subordinates s ON e.manager_id = s.id  
)  
SELECT * FROM subordinates ORDER BY depth, id;
```

Result:

id	name	manager_id	depth
2	Bob	1	1
3	Charlie	1	1
4	David	2	2
5	Eve	2	2
6	Frank	3	2

Recursive CTE Example - Graph Paths

Use case: social-network traversal in SQL

```
CREATE TABLE friendships (  
  person1 VARCHAR(255), person2 VARCHAR(255),  
  PRIMARY KEY (person1, person2));  
INSERT INTO friendships VALUES  
  ('Alice', 'Bob'), ('Bob', 'Charlie'), ('Charlie', 'David'),  
  ('Alice', 'Eve'), ('Eve', 'Frank'), ('Frank', 'David');
```

What the table looks like:

person1	person2
Alice	Bob
Alice	Eve
Bob	Charlie
Charlie	David
Eve	Frank
Frank	David

(6 rows)

Find paths from Alice to David:

```
WITH RECURSIVE paths AS (  
  SELECT person1, person2, 1 AS hops, ARRAY[person1, person2]::text[] AS path  
  FROM friendships WHERE person1 = 'Alice'  
  UNION ALL  
  SELECT p.person1, f.person2, p.hops + 1, p.path || f.person2::text  
  FROM paths p JOIN friendships f ON p.person2 = f.person1  
  WHERE NOT f.person2 = ANY(p.path) AND p.hops < 10  
)  
SELECT person1, person2, hops, path  
FROM paths WHERE person2 = 'David' ORDER BY hops;
```

Result:

person1	person2	hops	path
Alice	David	3	{Alice,Bob,Charlie,David}
Alice	David	3	{Alice,Eve,Frank,David}

(2 rows)

When Recursive SQL Helps

Recursive SQL is strong when:

- Data is already relational
- You need hierarchical reporting
- You need one specific traversal in SQL

Graph query languages help when:

- Traversal is the main workload
- Path patterns are variable-length
- Relationship types matter as much as entity attributes
- You want graph-shaped reasoning, not just table joins

This is where **Cypher** and **Apache AGE** become useful.

Per-session setup (official AGE pattern):

```
CREATE EXTENSION IF NOT EXISTS age;  
LOAD 'age';  
SET search_path = ag_catalog, "$user", public;
```

Create a graph:

```
SELECT * FROM ag_catalog.create_graph('movie_graph');
```

Typical AGE workflow:

1. Load AGE in the session
2. Create a graph
3. Create vertices and edges with Cypher
4. Query patterns with `MATCH`

How AGE Stores Graphs in PostgreSQL

Official AGE documentation shows that each graph gets its own PostgreSQL namespace.

Inspect created graphs:

```
SELECT * FROM ag_catalog.ag_graph;
```

Example output:

graphid	name	namespace
17026	movie_graph	movie_graph

(1 row)

Note: `graphid` is an internal PostgreSQL OID and will vary.

Inspect labels and their backing tables:

```
SELECT name, kind, relation
FROM ag_catalog.ag_label
WHERE relation::text LIKE 'movie_graph.%'
ORDER BY id;
```

Label types created:

name	kind	relation
_ag_label_vertex	v	movie_graph._ag_label_vertex
_ag_label_edge	e	movie_graph._ag_label_edge
Person	v	movie_graph."Person"
Movie	v	movie_graph."Movie"
ACTED_IN	e	movie_graph."ACTED_IN"
DIRECTED	e	movie_graph."DIRECTED"
(6 rows)		

Why this matters:

- AGE is not magic storage outside PostgreSQL.
- Graph labels are backed by PostgreSQL tables.
- You get graph modelling while staying inside Postgres.

What Is Cypher?

Cypher is a declarative query language for property graphs — think “SQL for graphs.” You describe the **pattern** you want; the engine finds it.

- Created by Neo4j, now an open standard (**openCypher**), also used by Apache AGE
- Patterns are drawn as **ASCII art**: `()` is a node, `-[]->` is a relationship
- Reads the way the graph looks: `(a)-[:KNOWS]->(b)`

Two mindsets:

- SQL: join tables on keys to *reconstruct* relationships
- Cypher: relationships already exist — you just *walk* them

Reading a Cypher Pattern

```
(p:Person {name:'Tom Hanks'})-[:ACTED_IN]->(m:Movie)
```

- `(p:Person)` — a node, variable `p`, label `Person`
- `{name:'Tom Hanks'}` — a property filter
- `-[:ACTED_IN]->` — a directed relationship of type `ACTED_IN`
- `(m:Movie)` — the node on the other end

The main clauses:

- `MATCH` find a pattern · `WHERE` filter · `RETURN` project
- `CREATE` / `MERGE` add data · `SET` update · `DELETE` remove

In AGE, Cypher Lives Inside SQL

Apache AGE runs Cypher **through a SQL function** — you stay in PostgreSQL:

```
SELECT * FROM cypher('graph_name', $$
    <your Cypher goes here>
$$) AS (col1 agtype, col2 agtype);
```

- `cypher(...)` takes the **graph name** and a Cypher block between `$$ ... $$`
- You must declare the returned columns, all typed `agtype` (AGE's value type)
- In native Neo4j you would just type the Cypher directly — no wrapper

Cypher Patterns and Data Model

Cypher is declarative: you describe the graph pattern you want.

Core pattern notation:

```
(p:Person {name: 'Tom Hanks'})  
(m:Movie {title: 'Sleepless in Seattle'})  
(p)-[:ACTED_IN]->(m)
```

Interpretation:

- `Person` and `Movie` are node labels
- `ACTED_IN` is a relationship type
- `{name: 'Tom Hanks'}` is a property map
- `->` shows relationship direction

Compare to Neo4j official Cypher examples:

- Node labels classify nodes
- Relationship types classify edges
- Properties store business meaning
- Path matching answers traversal questions

Apache AGE - Create a Movie Graph

The movie example clearly shows labels, typed relationships, and traversal.

Create all nodes:

```
SELECT * FROM cypher('movie_graph', $$  
    CREATE (:Person {name: 'Tom Hanks', born: 1956}),  
           (:Person {name: 'Meg Ryan', born: 1961}),  
           (:Person {name: 'Nora Ephron', born: 1941}),  
           (:Movie {title: 'Sleepless in Seattle', released: 1993})  
$$) AS (v agtype);
```

Create relationships:

```
SELECT * FROM cypher('movie_graph', $$  
  MATCH (tom:Person {name: 'Tom Hanks'}),  
        (meg:Person {name: 'Meg Ryan'}),  
        (nora:Person {name: 'Nora Ephron'}),  
        (movie:Movie {title: 'Sleepless in Seattle'})  
  CREATE (tom)-[:ACTED_IN {role: 'Sam Baldwin'}]->(movie),  
        (meg)-[:ACTED_IN {role: 'Annie Reed'}]->(movie),  
        (nora)-[:DIRECTED]->(movie)  
  RETURN movie  
$$) AS (movie agtype);
```

What the graph data means:

```
(:Person {name: 'Tom Hanks'})-[ACTED_IN]->(:Movie {title: 'Sleepless in Seattle'})  
(:Person {name: 'Meg Ryan'})-[ACTED_IN]->(:Movie {title: 'Sleepless in Seattle'})  
(:Person {name: 'Nora Ephron'})-[DIRECTED]->(:Movie {title: 'Sleepless in Seattle'})
```

Apache AGE - MATCH Queries

Official AGE **MATCH** patterns are built around nodes, labels, directions, and edge types.

Get all vertices:

```
SELECT * FROM cypher('movie_graph', $$  
  MATCH (v)  
  RETURN v  
$$) AS (v agtype);
```

Find all actors in Sleepless in Seattle:

```
SELECT * FROM cypher('movie_graph', $$
  MATCH (:Movie {title: 'Sleepless in Seattle'})<-[ACTED_IN]-(actor:Person)
  RETURN actor.name
$$) AS (actor_name agtype);
```

Find the director of the movie:

```
SELECT * FROM cypher('movie_graph', $$
  MATCH (director:Person)-[DIRECTED]->(:Movie {title: 'Sleepless in Seattle'})
  RETURN director.name
$$) AS (director_name agtype);
```

Result:

```
actor_name
-----
"Tom Hanks"
"Meg Ryan"
(2 rows)
```

Director result:

```
director_name
-----
"Nora Ephron"
(1 row)
```

Variable-Length Paths in AGE

AGE supports variable-length edges in Cypher — one of the biggest differences from plain SQL joins.

Create a social graph (KNOWS edges mirror the **friendships** table):

```
SELECT * FROM ag_catalog.create_graph('social_graph');
SELECT * FROM cypher('social_graph', $$
  CREATE (a:Person {name: 'Alice'}), (b:Person {name: 'Bob'}),
        (c:Person {name: 'Charlie'}), (d:Person {name: 'David'}),
        (e:Person {name: 'Eve'}), (f:Person {name: 'Frank'})
  CREATE (a)-[:KNOWS]->(b), (b)-[:KNOWS]->(c), (c)-[:KNOWS]->(d),
        (a)-[:KNOWS]->(e), (e)-[:KNOWS]->(f), (f)-[:KNOWS]->(d)
  CREATE (a)-[:WORKS_AT]->(co:Company {name: 'TechCorp'})
  CREATE (co)-[:HAS_OFFICE]->(:City {name: 'NYC'})
$$) AS (v agtype);
```

Two-hop traversal (exactly two **KNOWS** edges from Alice):

```
SELECT * FROM cypher('social_graph', $$  
  MATCH (:Person {name: 'Alice'})-[:KNOWS*2]->(friend)  
  RETURN friend.name  
$$) AS (friend_name agtype);
```

```
friend_name  
-----  
"Charlie"  
"Frank"  
(2 rows)
```

What this means:

- `*2` means exactly two relationships
- `*1..3` would mean between one and three relationships
- `*` (unbounded) follows the chain as far as it goes
- This is the graph equivalent of recursive path expansion — but the engine does the bookkeeping

"People you may know" — friends-of-friends not already connected:

```
SELECT * FROM cypher('social_graph', $$
  MATCH (a:Person {name: 'Alice'})-[:KNOWS*1..2]->(c:Person)
  WHERE c <> a AND NOT exists((a)-[:KNOWS]->(c))
  RETURN DISTINCT c.name
$$) AS (suggestion agtype);
```

Returns **Charlie** and **Frank** — the 2-hop contacts.

Note: AGE needs `NOT exists((a)-[:KNOWS]->(c))` ; a bare `NOT (a)-[:R]->(c)` won't parse.

Rosetta Stone: Same Question, Two Ways

Ask the **same question** in recursive SQL and in Cypher, side by side. We mirror the `employees` org chart into `org_graph`:

```
SELECT * FROM ag_catalog.create_graph('org_graph');
SELECT * FROM cypher('org_graph', $$
  CREATE (a:Emp {name:'Alice'}), (b:Emp {name:'Bob'}), (c:Emp {name:'Charlie'}),
         (d:Emp {name:'David'}), (e:Emp {name:'Eve'}), (f:Emp {name:'Frank'})
  CREATE (a)-[:MANAGES]->(b), (a)-[:MANAGES]->(c),
         (b)-[:MANAGES]->(d), (b)-[:MANAGES]->(e), (c)-[:MANAGES]->(f)
$$) AS (v agtype);
```

Q1: "All of Alice's reports, at any depth?" — Recursive SQL

You manage the expansion yourself:

```
WITH RECURSIVE subordinates AS (  
  SELECT id, name, manager_id, 1 AS depth FROM employees  
  WHERE manager_id = (SELECT id FROM employees WHERE name = 'Alice')  
  UNION ALL  
  SELECT e.id, e.name, e.manager_id, s.depth + 1  
  FROM employees e JOIN subordinates s ON e.manager_id = s.id  
)  
SELECT name, depth FROM subordinates ORDER BY depth, name;
```

Q1 — Cypher

You state the pattern; the engine expands it:

```
SELECT * FROM cypher('org_graph', $$  
  MATCH p = (:Emp {name: 'Alice'})-[:MANAGES*]->(report:Emp)  
  RETURN report.name, length(p) AS depth  
  ORDER BY length(p), report.name  
  $$) AS (report_name agtype, depth agtype);
```

Both return Bob/Charlie at depth 1 and David/Eve/Frank at depth 2.

Q2: "What routes connect Alice to David?" — Recursive SQL

Track the visited set, cap the depth, build the path array:

```
WITH RECURSIVE paths AS (  
  SELECT person1, person2, 1 AS hops, ARRAY[person1, person2]::text[] AS path  
  FROM friendships WHERE person1 = 'Alice'  
  UNION ALL  
  SELECT p.person1, f.person2, p.hops + 1, p.path || f.person2::text  
  FROM paths p JOIN friendships f ON p.person2 = f.person1  
  WHERE NOT f.person2 = ANY(p.path) AND p.hops < 10  
)  
SELECT path, hops FROM paths WHERE person2 = 'David' ORDER BY hops;
```

Q2 — Cypher

One variable-length pattern; cycle handling is built in:

```
SELECT * FROM cypher('social_graph', $$  
  MATCH p = (:Person {name: 'Alice'})-[:KNOWS*]->(:Person {name: 'David'})  
  RETURN nodes(p) AS path_nodes, length(p) AS hops  
  ORDER BY length(p)  
 $$) AS (path_nodes agtype, hops agtype);
```

Both find: Alice→Bob→Charlie→David and Alice→Eve→Frank→David.

The takeaway:

	Recursive SQL	Cypher
Style	Imperative bookkeeping	Declarative pattern
You write	anchor + recursive step + visited-set + depth cap	the shape you want
Cycles	You prevent them	Engine handles them
Best when	data is relational, one-off traversal	traversal <i>is</i> the workload

Same data, same answer — Cypher just removes the plumbing.

Knowledge Graph Modelling

Graphs are especially useful when the domain is mostly relationships.

A science-fiction knowledge graph:

```
(Asimov)-[:WROTE]->(Foundation)-[:IN_GENRE]->(Science Fiction)
(Asimov)-[:BORN_IN]->(Russia)           (Asimov)-[:INFLUENCED]->(Gibson)
(Gibson)-[:WROTE]->(Neuromancer)-[:CITES]->(I, Robot)
```

Entities become nodes; every fact becomes a typed, queryable edge.

Build the knowledge graph — first the nodes:

```
SELECT * FROM ag_catalog.create_graph('books_graph');
SELECT * FROM cypher('books_graph', $$
    CREATE (:Author {name:'Isaac Asimov'}), (:Author {name:'William Gibson'}),
           (:Book {title:'Foundation', year:1951}),
           (:Book {title:'I, Robot', year:1950}),
           (:Book {title:'Neuromancer', year:1984}),
           (:Country {name:'Russia'}), (:Genre {name:'Science Fiction'})
    $$) AS (v agtype);
```

The full graph in `code/week5.sql` adds Clarke, another book, and `HAS_TOPIC` edges.

Then connect them with typed edges:

```
SELECT * FROM cypher('books_graph', $$  
  MATCH (asimov:Author {name:'Isaac Asimov'}), (gibson:Author {name:'William Gibson'}),  
        (foundation:Book {title:'Foundation'}), (irobot:Book {title:'I, Robot'}),  
        (neuro:Book {title:'Neuromancer'}), (russia:Country {name:'Russia'}),  
        (scifi:Genre {name:'Science Fiction'})  
  CREATE (asimov)-[:WROTE]->(foundation), (asimov)-[:WROTE]->(irobot),  
        (gibson)-[:WROTE]->(neuro), (asimov)-[:BORN_IN]->(russia),  
        (foundation)-[:IN_GENRE]->(scifi), (irobot)-[:IN_GENRE]->(scifi),  
        (neuro)-[:IN_GENRE]->(scifi),  
        (asimov)-[:INFLUENCED]->(gibson), (neuro)-[:CITES]->(irobot)  
$$) AS (v agtype);
```

Graph RAG: Retrieval over a Knowledge Graph

Why graphs matter for AI / retrieval:

- Facts are explicit, typed edges — not text the model must re-read
- Multi-hop questions are answered by *traversal*, not similarity guessing
- The retrieved **path is the reasoning chain** — explainable by construction

(1) Multi-hop question: *"Which sci-fi books were written by authors born in Russia?"*

```
SELECT * FROM cypher('books_graph', $$
  MATCH (:Country {name: 'Russia'}) <-[:BORN_IN]-(a:Author)
    -[:WROTE]->(b:Book)-[:IN_GENRE]->(:Genre {name: 'Science Fiction'})
  RETURN a.name AS author, b.title AS book
$$) AS (author agtype, book agtype);
```

author		book
Isaac Asimov		Foundation
Isaac Asimov		I, Robot

One pattern crosses Country → Author → Book → Genre. In SQL, that's four joins.

(2) Connection retrieval: *"How is Asimov related to Gibson?"*

```
SELECT * FROM cypher('books_graph', $$
  MATCH p = (a:Author {name:'Isaac Asimov'})-[*1..4]-(b:Author {name:'William Gibson'})
  RETURN nodes(p) AS path_nodes, length(p) AS hops
  ORDER BY length(p) LIMIT 1
$$) AS (path_nodes agtype, hops agtype);
```

- Returns the shortest connecting path: `Asimov -[:INFLUENCED]-> Gibson` (1 hop).
- AGE 1.6.0 has no `shortestPath()` — we emulate it with `*1..n` + `ORDER BY length(p) LIMIT 1`. The returned path is the explanation.

(3) Context retrieval for grounding an LLM: pull the neighborhood of an entity.

```
SELECT * FROM cypher('books_graph', $$
  MATCH (b:Book {title:'Neuromancer'})-[*1..2]-(ctx)
  RETURN DISTINCT label(ctx) AS kind, ctx
$$) AS (kind agtype, ctx agtype);
```

- Returns the 1–2 hop subgraph around *Neuromancer*: its author, genre, topic, and the book it cites — the **structured context** you hand to the model.
- This is the core of Graph RAG: retrieve a *subgraph*, not a blob of text.

(4) Aggregated facts: assemble a per-entity bundle with `collect()` .

```
SELECT * FROM cypher('books_graph', $$
  MATCH (a:Author {name:'Isaac Asimov'})-[:WROTE]->(b:Book)
  WITH a, collect(b.title) AS books, count(*) AS book_count
  RETURN a.name AS author, book_count, books
$$) AS (author agtype, book_count agtype, books agtype);
```

author	book_count	books
"Isaac Asimov"	2	["Foundation", "I, Robot"]
(1 row)		

Vector RAG vs Graph RAG

	Vector RAG	Graph RAG
Retrieval	nearest text chunks	typed multi-hop traversal
"Why this answer?"	opaque similarity	the path is the evidence
Multi-hop	brittle	native (<code>-[:X]->-[:Y]-></code>)
Best for	fuzzy semantic recall	linked, factual reasoning

In practice they combine: vectors find the **entry node**, the graph supplies the **connected facts**.

Graph Thinking for AI

Reasoning is path-finding over typed relationships:

```
Alice -[:WORKS_AT]-> TechCorp -[:HAS_OFFICE]-> NYC
```

```
SELECT * FROM cypher('social_graph', $$  
  MATCH (:Person {name: 'Alice'})-[:WORKS_AT]->(co)-[:HAS_OFFICE]->(city)  
  RETURN city.name  
$$) AS (city_name agtype);
```

Result: "NYC" .

Teaching point: the graph answers *and* shows its work — every hop is a stated fact.

Moving to Neo4j: What You Already Know

Cypher is **the same language** in Neo4j and AGE, so most skills transfer directly:

`MATCH` , `CREATE` , `WHERE` , `RETURN` , variable-length `*1..3` , `MERGE` .

What is actually simpler in Neo4j:

- No `cypher('graph', $$ ... $$) AS (col agtype)` wrapper — you write Cypher directly
- Results come back as real nodes and relationships, not `agtype` text
- You connect via the **Neo4j Browser**, `cypher-shell` , or **Bolt** drivers (Python / JS)

Moving to Neo4j: What's New to Learn

You know Cypher; Neo4j adds capabilities AGE 1.6.0 leaves out:

- **Real** `shortestPath()` / `allShortestPaths()` — no `ORDER BY length(p) LIMIT 1` trick
- **Fuller Cypher:** `UNWIND` , `FOREACH` , `CASE` , list & pattern comprehensions, `$parameters`
- **Constraints & indexes:** `CREATE CONSTRAINT` , uniqueness, indexes for performance
- **Bulk import:** `LOAD CSV` , `neo4j-admin import`
- **Procedures & libraries:** `CALL` , the **APOC** toolkit, and **GDS** (PageRank, community detection, centrality)

Summary

Week 5 covered:

- Property graph data model: nodes, labels, relationships, properties, paths
- Relational graph storage with adjacency lists
- Recursive CTEs for hierarchies and graph-like traversal
- Where recursive SQL becomes awkward
- Cypher background: reading patterns and clauses, and how AGE wraps it in SQL
- Apache AGE setup, graph storage, pattern matching, and variable-length paths
- Rosetta stone: the same question in recursive SQL vs Cypher
- Knowledge graphs and Graph RAG retrieval for AI
- Moving to Neo4j: what transfers and what is new to learn

You can now:

- Explain the property-graph model
- Represent a graph in relational tables
- Write recursive SQL traversals
- Translate a traversal between recursive SQL and Cypher
- Create and query graphs in Apache AGE
- Design a knowledge graph and run Graph-RAG retrieval queries

Official runnable examples:

- See `code/week5.sql` (PostgreSQL + AGE) — every query in these slides runs there

Final week (Week 6):

- ACID transactions
- Stored procedures
- Spark and distributed computing
- Databricks and Lakehouse

EXERCISES (30 minutes)

Exercise 1: Recursive CTE - Hierarchies

Create sample data:

```
CREATE TABLE org_chart (  
  emp_id INT PRIMARY KEY, name VARCHAR(255),  
  manager_id INT REFERENCES org_chart(emp_id));  
INSERT INTO org_chart VALUES  
  (1, 'Alice', NULL), (2, 'Bob', 1), (3, 'Charlie', 1),  
  (4, 'David', 2), (5, 'Eve', 2), (6, 'Frank', 3);
```

Query: Find all subordinates with levels:

```
WITH RECURSIVE hierarchy AS (  
    SELECT emp_id, name, manager_id, 1 AS level  
    FROM org_chart WHERE manager_id IS NULL  
    UNION ALL  
    SELECT e.emp_id, e.name, e.manager_id, h.level + 1  
    FROM org_chart e JOIN hierarchy h ON e.manager_id = h.emp_id  
)  
SELECT *, REPEAT('  ', level - 1) || name AS hierarchy_view  
FROM hierarchy ORDER BY level, emp_id;
```

Exercises:

1. Find all direct and indirect reports of Bob.
2. Find the manager chain for Frank.
3. Find the maximum depth of the organization.

Exercise 2: Recursive CTE - Graph Paths

Create friendship data:

```
CREATE TABLE friendships (  
  person VARCHAR(255), friend VARCHAR(255),  
  PRIMARY KEY (person, friend));  
INSERT INTO friendships VALUES  
  ('Alice', 'Bob'), ('Bob', 'Charlie'), ('Charlie', 'David'),  
  ('Alice', 'Eve'), ('Eve', 'Frank'), ('Frank', 'David');
```

Find paths from Alice to David:

```
WITH RECURSIVE paths AS (  
  SELECT person, friend, 1 AS hops, ARRAY[person, friend]::text[] AS path  
  FROM friendships WHERE person = 'Alice'  
  UNION ALL  
  SELECT p.person, f.friend, p.hops + 1, p.path || f.friend::text  
  FROM paths p JOIN friendships f ON p.friend = f.person  
  WHERE NOT f.friend = ANY(p.path) AND p.hops < 10  
)  
SELECT person, friend, hops, path  
FROM paths WHERE friend = 'David' ORDER BY hops;
```

Exercises:

1. Find the shortest path from Alice to David.
2. Find all people two hops away from Alice.
3. Add a cycle and prevent infinite recursion.

Exercise 3: Apache AGE - Setup and Inspect the Graph Catalog

Session setup:

```
CREATE EXTENSION IF NOT EXISTS age;  
LOAD 'age';  
SET search_path = ag_catalog, "$user", public;
```

Create graph and inspect it:

```
SELECT * FROM ag_catalog.create_graph('exercise_graph');  
SELECT * FROM ag_catalog.ag_graph;  
SELECT name, kind, relation  
FROM ag_catalog.ag_label  
WHERE relation::text LIKE 'exercise_graph.%'  
ORDER BY id;
```

Exercises:

1. Create a `Person` vertex label by inserting a person with Cypher.
2. Create a `KNOWS` edge label by linking two people.
3. Inspect `ag_label1` again and explain what changed.

Exercise 4: Apache AGE - Movie Graph Queries

Create the movie graph:

```
SELECT * FROM ag_catalog.create_graph('movies');
SELECT * FROM cypher('movies', $$
    CREATE (:Person {name: 'Tom Hanks'})-[:ACTED_IN]->(:Movie {title: 'Sleepless in Seattle'})
$$) AS (v agtype);
SELECT * FROM cypher('movies', $$
    MATCH (m:Movie {title: 'Sleepless in Seattle'})
    CREATE (:Person {name: 'Meg Ryan'})-[:ACTED_IN]->(m),
           (:Person {name: 'Nora Ephron'})-[:DIRECTED]->(m)
$$) AS (v agtype);
```

Exercises:

1. Find all actors in `Sleepless in Seattle`.
2. Find the director of the movie.
3. Add another movie and find all movies Tom Hanks acted in.

Exercise 5: Knowledge Graph + Graph RAG

Create a books graph:

```
SELECT * FROM ag_catalog.create_graph('books');  
SELECT * FROM cypher('books', $$  
  CREATE (:Author {name: 'Isaac Asimov'})-[:WROTE]->(:Book {title: 'Foundation', year: 1951})  
$$) AS (v agtype);
```

Build it out:

1. Add a **Country** node and a **BORN_IN** relationship.
2. Add a **Genre** node and an **IN_GENRE** relationship.
3. Add a second author + book, and an **INFLUENCED** edge between the authors.

Exercise 5 (cont.): Graph-RAG Retrieval

On the books graph you just built:

4. Multi-hop: return all sci-fi books by authors born in Russia.

5. Context: return the 1–2 hop neighborhood around `Foundation`

(hint: `MATCH (b:Book {title:'Foundation'})-[*1..2]-(ctx) RETURN DISTINCT label(ctx), ctx`).

6. Connection: find the shortest path between your two authors

(hint: `*1..4` + `ORDER BY length(p) LIMIT 1` — AGE has no `shortestPath()`).

Exercise 6: Rosetta - Translate Between SQL and Cypher

Same question, two engines. Using the `org_graph` from the slides:

1. Write the *manager chain for Frank* (Frank up to the CEO) as a recursive CTE on `employees`, then as a Cypher query on `org_graph`
(hint: `MATCH p = (boss:Emp)-[:MANAGES*]->(:Emp {name:'Frank'})`).
2. Count each person's total reports (any depth) in both SQL and Cypher.
3. Which version was easier to write? Which would you trust on a 10-hop question?

Reflection

1. When would you choose recursive SQL instead of a graph query language?
2. When does a relationship deserve to become an edge instead of a foreign key?
3. Why is the property graph model a better fit for some AI and knowledge tasks?
4. Design a graph for one of these domains:
 - recommendations
 - fraud detection
 - scientific citations

What are the node labels, relationship types, and key properties?

Challenge: Find Influencers in AGE

Graph structure:

```
(User)-[:FOLLOWS]->(User)  
(User)-[:LIKES]->(Post)  
(Post)-[:CREATED_BY]->(User)
```

Goal: return users with the most followers and likes.

AGE-compatible starting point:

```
SELECT * FROM cypher('social_graph', $$  
  MATCH (u:User)<-[f:FOLLOWS]-(:User)  
  RETURN u.name, COUNT(f) AS followers  
  ORDER BY COUNT(f) DESC LIMIT 10  
$$) AS (user_name agtype, follower_count agtype);
```

Extend it to also count likes on the user's posts.